

Document

CONTENTS

Autoregressive Token-Based TTS

Non-Autoregressive TTS: Diffusion and Flow Matching

Neural Vocoder

Zero-Shot Voice Cloning from Short Reference Audio

Streaming TTS: Low Time-to-First-Byte and Quality Trade-Offs

Multilingual Text and Embedded Foreign Words

Architecture Comparison Summary

Autoregressive Token-Based TTS

Mechanism

Autoregressive (AR) TTS models treat speech synthesis as a language modeling problem: predict the next token conditioned on all previous tokens. Tokens can be:

- **Discrete codec tokens** (e.g., EnCodec, DAC): 1,024-vocabulary indices from residual vector quantization [praveentn](#)
- **Continuous acoustic features**: mel-spectrogram frames or latent vectors [alphaxiv](#)

Key architectures:

- **VALL-E** (Microsoft, 2023): AR transformer predicts first RVQ codebook (coarse prosody + speaker identity), then non-autoregressive model fills in remaining codebooks in parallel [praveentn](#)

- **Voicebox** (Meta, 2024): AR codec LM trained on 60K hours of multilingual speech; supports zero-shot voice cloning from 3-second reference [localaimaster](#)
- **dots.tts** (2026): 2B-parameter continuous AR model with flow-matching head for high-fidelity synthesis [alphaxiv](#)

Streaming Capability

Native streaming: AR models emit tokens sequentially, allowing TTS to begin playback as soon as first tokens are generated. This enables low time-to-first-byte (TTFB) for conversational agents. [praveentn](#)

Latency characteristics:

- **Sequential decoding:** Each token depends on previous tokens, limiting parallelization [voicemodeler](#)
- **Exposure bias:** Training uses teacher forcing (ground-truth previous tokens), but inference uses model predictions, causing error accumulation over long sequences [alphaxiv](#)
- **Typical RTF:** 0.5–2.0× real-time (slower than real-time without optimization) [arxiv](#)

Typical Quality

AR models achieve **high naturalness** but struggle with:

- **Long-form coherence:** Errors compound over 10+ second utterances, causing word repetitions or skipped phonemes [voicemodeler](#)
- **Prosody control:** Implicit duration modeling via token sequence length; hard to explicitly control speaking rate [localaimaster](#)

PROPERTY	AUTOREGRESSIVE TTS
Streaming output	Yes (native) praveentn
Zero-shot voice cloning	Yes (3-sec reference) arxiv
Inference speed	Slow (sequential) alphaxiv
Prosody naturalness	High (implicit duration) studocu
Long-form stability	Moderate (error accumulation) alphaxiv

Non-Autoregressive TTS: Diffusion and Flow Matching

Diffusion-Based TTS

Diffusion models (e.g., Grad-TTS, NaturalSpeech, Voicebox) generate speech by iteratively denoising a Gaussian noise signal conditioned on text. The model learns to reverse a forward noising process over 50–200 steps. [localaimaster](#)

Key architectures:

- **NaturalSpeech 3** (2024): Latent diffusion on mel-spectrograms with factorized codec for zero-shot TTS [arxiv](#)
- **Voicebox** (Meta, 2024): Diffusion transformer (DiT) with text-guided speech infilling; supports multilingual synthesis [localaimaster](#)

Flow Matching TTS

Flow matching (FM) reframes diffusion as learning a velocity field that transforms noise into data along an optimal transport path. This enables faster convergence and fewer inference steps (10–20 vs. 50–200 for diffusion). [localaimaster](#)

Key architectures:

- **F5-TTS** (2025): Fully non-autoregressive flow-matching model with Diffusion Transformer (DiT) backbone; achieves RTF of 0.016 (60× faster than real-time) [localaimaster](#)
- **ARCHI-TTS** (2025): Flow-matching aligner-encoder-decoder with semantic aligner for text-speech alignment [arxiv](#)
- **DiFlow-TTS** (2026): Discrete flow matching on factorized codec tokens; combines AR quality with NAR speed [alphaxiv](#)

How Flow Matching Works

Flow matching learns a conditional velocity field $(u_\theta(X_t, t, z))$ that transforms noise $(X_0 \sim \mathcal{N}(0, I))$ into target speech (X_1) along a path $(X_t = tX_1 + (1-t)X_0)$. The loss is: [studocu](#)

$$\| \mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, X_0, X_1} \left[\| u_{\theta}(X_t, t, z) - (X_1 - X_0) \|^2 \right]$$

At inference, an ODE solver integrates from $t=0$ to $t=1$ to generate speech. [studocu](#)

Advantages over diffusion:

- **Faster training:** Converges in fewer steps due to straighter ODE paths [localaimaster](#)
- **Fewer inference steps:** 10–20 ODE steps vs. 50–200 diffusion steps [localaimaster](#)
- **Better alignment:** Flow matching with optimal transport (FM-OT) learns text-speech alignment implicitly [localaimaster](#)

Streaming Capability

Non-streaming by default: NAR models generate entire utterance in parallel, requiring full text input before synthesis begins. This creates a latency bottleneck for conversational TTS. [alphaxiv](#)

Workarounds:

- **Chunked generation:** Split text into sentences/clauses; run flow matching on each chunk independently [localaimaster](#)
- **Sway Sampling (F5-TTS):** Inference-time sampling strategy that prioritizes early flow steps, enabling faster startup with fewer ODE steps [localaimaster](#)

PROPERTY	NON-AUTOREGRESSIVE (FLOW MATCHING)
Streaming output	No (parallel generation) studocu
Zero-shot voice cloning	Yes (audio prompt + text) arxiv
Inference speed	Fast (10–20 ODE steps) studocu
Prosody naturalness	High (implicit duration via padding) arxiv
Long-form stability	High (no error accumulation) alphaxiv

Neural Vocoder

Role in TTS Pipeline

Neural vocoders convert acoustic features (mel-spectrograms or codec latents) into raw waveforms. They are the final stage in most TTS pipelines, determining audio fidelity and artifact levels. [praveentn](#)

Two families:

1. Waveform-domain vocoders:

- **WaveNet** (DeepMind, 2016): Autoregressive dilated causal convolutions; high quality but slow (RTF < 1× on CPU) [praveentn](#)
- **HiFi-GAN** (2020): Non-autoregressive GAN with transposed convolutions; real-time or faster on GPU; near-WaveNet quality [praveentn](#)
- **BigVGAN** (2022): Universal neural vocoder trained on 200K hours; supports multiple sampling rates [themoonlight](#)

1. Codec-based vocoders:

- **SoundStream** (Google, 2021): RVQ-based neural audio codec; 3 kbps speech compression with better quality than Opus at 12 kbps [praveentn](#)
- **EnCodec** (Meta, 2022): Multiscale spectrogram discriminator + loss balancer; supports 24 kHz mono and 48 kHz stereo [praveentn](#)

Codec Language Models

Codec LMs (e.g., VALL-E, Voicebox) collapse the acoustic model and vocoder into a single conditional language model trained on quantized audio tokens. This enables: [praveentn](#)

- **Zero-shot voice cloning:** Given 3-second reference audio, model generates codec tokens in that speaker's voice [voicemodeler](#)
- **End-to-end training:** No separate mel-spectrogram prediction + vocoder stages; model learns alignment implicitly [voicemodeler](#)

Classical pipeline: text → phonemes → mel spectrogram → waveform

Codec LM pipeline: text + speaker tokens → codec tokens → waveform

Zero-Shot Voice Cloning from Short Reference Audio

How It Works

Zero-shot voice cloning requires only a short reference clip (3–10 seconds) from an unseen speaker. The model:

- 1. Encodes speaker identity:** Extracts speaker embedding from reference audio (e.g., d-vector, x-vector, or codec latents) [praveentn](#)
- 2. Conditions generation:** Uses speaker embedding as additional input to TTS model, guiding synthesis to match reference timbre and prosody [voicemodeler](#)
- 3. Generates speech:** Synthesizes target text in cloned voice without fine-tuning [praveentn](#)

Key Systems

SYSTEM	REFERENCE LENGTH	ARCHITECTURE	ZERO-SHOT QUALITY
VALL-E (Microsoft, 2023)	3 seconds	AR codec LM (first codebook) + NAR (remaining codebooks) praveentn	Human parity on LibriSpeech/VCTK voicemodeler
Voicebox (Meta, 2024)	3 seconds	Flow-matching DiT with speech infilling arxiv	Multilingual zero-shot (100K hours training) studocu
F5-TTS (2025)	5–10 seconds	Flow matching + DiT + ConvNeXt text refinement studocu	Seamless code-switching; RTF 0.016 studocu
CosyVoice 2 (2025)	5 seconds	Semantic tokens + Qwen LLM + CFM arxiv	High-fidelity multilingual synthesis arxiv

Quality Trade-Offs

- Instant cloning (1–5 min audio):** Fast setup, but may miss subtle voice characteristics (regional accent, laryngeal quality) [praveentn](#)
- Professional cloning (30+ min audio):** Fine-tuned model captures idiosyncrasies; requires dedicated training run [praveentn](#)

Streaming TTS: Low Time-to-First-Byte and Quality Trade-Offs

How Streaming TTS Achieves Low TTFB

Streaming TTS begins audio playback before the full response text is available. Key techniques:

- 1. **Sentence/clause chunking:** Split LLM output at sentence boundaries (periods, question marks); flush each sentence to TTS immediately
- 2. **Predict-ahead synthesis:** TTS model starts phoneme generation from first few tokens using lookahead (e.g., 120 chars = ~75 ms on Flash v2.5)
- 3. **WebSocket streaming:** Persistent bi-directional connection delivers audio chunks incrementally; playback starts before synthesis completes

Quality Trade-Offs

STRATEGY	CHUNK SIZE	FIRST-BYTE LATENCY	PROSODY QUALITY	USE CASE
Sentence boundary	10–50 words	100–200 ms	Natural (complete intonation)	Default for most agents
Clause boundary	5–15 words	75–150 ms	Slightly choppy	Low-latency agents
Fixed token count	24–96 tokens	50–100 ms	Robotic (mid-clause breaks)	Experimental

Trade-off: Smaller chunks reduce TTFB but risk choppy prosody (mid-sentence breaks, unnatural pauses). Larger chunks sound more natural but delay first audio.

Production tuning: Bias toward smaller chunks early in utterance (user is waiting) and larger chunks later (audio already playing).

Multilingual Text and Embedded Foreign Words

The Challenge

Multilingual TTS must handle:

- **Code-switching:** Alternating languages within a single utterance (e.g., "I need to book a flight *بكره*" mixing English and Arabic)
- **Foreign proper nouns:** English drug names inside Arabic sentences (e.g., "الجرعة الموصى بها" هي *ibuprofen*) [praveentn](#)
- **Language detection:** Identifying which language each word belongs to for correct phoneme lookup [praveentn](#)

Why Language Detection on Code-Mixed Text Is Unreliable

1. **Orthographic ambiguity:** Many languages share scripts (e.g., English and Arabic both use Latin/Arabic scripts, but loanwords may be written in either)
2. **Phonetic borrowing:** Foreign words are often pronounced with native phonology (e.g., "ibuprofen" → Arabic phonemes /ibru:fen/)
3. **Context-dependent pronunciation:** Same word may be pronounced differently based on surrounding language (e.g., French "restaurant" in English vs. French context) [praveentn](#)
4. **Sparse training data:** Most TTS corpora are monolingual; code-switched data is rare and expensive to collect

Modern Approaches

APPROACH	DESCRIPTION	LIMITATIONS
Multilingual training	Train on mixed-language corpora (e.g., Emilia 95K hours, LEMAS 150K hours) arxiv	Requires massive CSW datasets; most public datasets are monolingual
Language-adaptive layers	Add language-specific adapter modules to shared encoder	Increases model complexity; requires language tags at inference
CSW-aware LMs	Train external LMs on code-switched text; use shallow fusion	Limited CSW text corpora; most web text is monolingual
Token-level language ID	Predict language ID per token alongside transcription	Adds output dimension; requires token-level language annotations

State-of-the-art: F5-TTS and Voicebox support seamless code-switching via flow matching on multilingual datasets (100K+ hours), but WER degrades 20–50% on mixed-language utterances vs. monolingual. [studocu](#)

Example: English Drug Name in Arabic Sentence

Input: "مرتين يوميًا *ibuprofen* الجرعة الموصى بها هي" (The recommended dose is *ibuprofen* twice daily)

Failure modes:

- **Mispronunciation:** TTS reads "ibuprofen" with Arabic phonology (/ibru:fen/) instead of English (/ˌaɪbjuːˈprɒfən/) [praveentn](#)
- **Language ID error:** Model assumes entire sentence is Arabic; applies Arabic G2P rules to English word [praveentn](#)
- **Prosody mismatch:** English word inserted with Arabic intonation contour, sounding unnatural

Workarounds:

- **Explicit language tagging:** Use SSML `<lang xml:lang="en-US">ibuprofen</lang>` to force English pronunciation [praveentn](#)
- **Phonetic transcription:** Provide IPA or ARPABET for foreign words (e.g., "ibuprofen" → /ˌaɪbjuːˈprɒfən/) [praveentn](#)
- **Multilingual G2P:** Train grapheme-to-phoneme model on code-switched data to learn context-dependent pronunciation

Architecture Comparison Summary

PROPERTY	AUTOREGRESSIVE (VALL-E, VOICEBOX)	NON-AUTOREGRESSIVE (F5-TTS, FLOW MA
Streaming output	Yes (native) praveentn	No (parallel generation) studocu
Zero-shot voice cloning	Yes (3-sec reference) arxiv	Yes (audio prompt + text) arxiv

PROPERTY	AUTOREGRESSIVE (VALL-E, VOICEBOX)	NON-AUTOREGRESSIVE (F5-TTS, FLOW MATCHING)
Inference speed	Slow (sequential decoding) alphaxiv	Fast (10–20 ODE steps) studocu
Prosody naturalness	High (implicit duration) studocu	High (implicit via padding) arxiv
Long-form stability	Moderate (error accumulation) alphaxiv	High (no error accumulation) alphaxiv
Multilingual support	Moderate (requires multilingual training)	High (flow matching generalizes better) arxiv
Code-switching	Poor (language ID ambiguity)	Moderate (F5-TTS supports seamless CSV) studocu

Key papers:

- **VALL-E:** Wang et al. (2023), "Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers" [praveentn](#)
- **Voicebox:** Le et al. (2024), "Voicebox: Text-guided multilingual universal speech generation at scale" [localaimaster](#)
- **F5-TTS:** Chen et al. (2025), "F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching" [localaimaster](#)
- **Flow matching:** Lipman et al. (2022), "Flow Matching for Generative Modeling" [studocu](#)

Production systems in 2026 increasingly use **hybrid architectures**: AR for streaming partials + NAR flow matching for high-fidelity final synthesis, or multilingual flow-matching models with language-adaptive layers for code-switching. [arxiv](#)